

Федеральное государственное автономное образовательное учреждение
высшего образования
“Национальный исследовательский университет ИТМО”

Дисциплина: Облачные и туманные вычисления
Лабораторная работа №2: Миграция в Kubernetes (Переход к Cloud-Native)

Работу выполнил:
Рудкевич Илья Александрович
студент группы Р3406

Ход работы

Исходный код: <https://github.com/gfxv/cloud-and-fog-computing-lab2>

Создание кластера

```
> kind create cluster --name lab2
Creating cluster "lab2" ...
  ✓ Ensuring node image (kindest/node:v1.35.0) 🚩
  ✓ Preparing nodes 📦
  ✓ Writing configuration 📄
  ✓ Starting control-plane 🚦
  ✓ Installing CNI 🚧
  ✓ Installing StorageClass 💾
Set kubectl context to "kind-lab2"
You can now use your cluster with:

kubectl cluster-info --context kind-lab2

Have a question, bug, or feature request? Let us know! https://kind.sigs.k8s.io/#community 😊
```

Создание манифестов K8S

Deployment конфиг для основного сервиса

```
k8s/app-deployment.yml > spec > template > spec > contain
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: python-app
5  spec:
6    replicas: ${APP_REPLICAS}
7    selector:
8      matchLabels:
9        app: python-app
10   template:
11     metadata:
12       labels:
13         app: python-app
14     spec:
15       containers:
16         - name: python-app
17           image: ${APP_IMAGE}
18           imagePullPolicy: Never
19           ports:
20             - containerPort: ${APP_PORT}
21           envFrom:
22             - configMapRef:
23               name: app-config
24             - secretRef:
25               name: app-secrets
```

Service конфиг для основного сервиса

```
k8s/app-service.yml > spec > ports
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: python-app-service
5  spec:
6    type: NodePort
7    selector:
8      app: python-app
9    ports:
10     - port: ${APP_PORT}
11       targetPort: ${APP_PORT}
12       nodePort: ${APP_NODE_PORT}
13
```

Deployment конфиг для Redis

```
k8s/redis-deployment.yml > spec > template > spec > containers > ports

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: redis
5  spec:
6    replicas: 1
7    selector:
8      matchLabels:
9        app: redis
10   template:
11     metadata:
12       labels:
13         app: redis
14     spec:
15       containers:
16         - name: redis
17           image: redis:7-alpine
18           envFrom:
19             - secretRef:
20                 name: app-secrets
21           command:
22             [
23               "sh",
24               "-c",
25               "redis-server --appendonly yes --requirepass $REDIS_PASSWORD",
26             ]
27           ports:
28             - containerPort: ${REDIS_PORT}
```

Service конфиг для Redis

```
k8s/redis-service.yml > spec > ports

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: redis
5  spec:
6    type: ClusterIP
7    selector:
8      app: redis
9    ports:
10     - port: ${REDIS_PORT}
11       targetPort: ${REDIS_PORT}
```

Конфигурация

```
k8s/config.yml > data
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: app-secrets
5  type: Opaque
6  stringData:
7    REDIS_PASSWORD: password
8  ---
9  apiVersion: v1
10 kind: ConfigMap
11 metadata:
12   name: app-config
13 data:
14   REDIS_HOST: redis
15   REDIS_PORT: "${REDIS_PORT}"
16   REDIS_DB: "0"
```

Развертывание

Соберем docker образ и загрузим его в кластер

```
> docker build -t lab2 .
[+] Building 0.6s (10/10) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 408B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd
=> [internal] load build context
=> => transferring context: 136.56kB
=> CACHED [2/5] WORKDIR /api
=> CACHED [3/5] COPY api/requirements.txt ./
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY api/ ./
=> exporting to image
=> => exporting layers
=> => writing image sha256:51afaa3a314586b2eda848ef32e25231956903c5e0
=> => naming to docker.io/library/lab2
```

```
> kind load docker-image lab2 --name lab2
Image: "lab2" with ID "sha256:51afaa3a314586b2eda848ef32e25231956903c5e0c03188380e9ea2029e1682" not yet present on node "lab2-control-plane", loading...
lab2 git-master
```

Применим минифесты с подстановкой переменных окружения из .env:

```
> make env
. .env && for f in k8s/*.yaml; do \
    envsubst < "$f" | kubectl apply -f -; \
done
deployment.apps/app created
service/python-app-service unchanged
secret/app-secrets configured
configmap/app-config unchanged
deployment.apps/redis unchanged
service/redis unchanged
```

Все поды запустились

```
> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
app-7d79574b7c-6sn2p	1/1	Running	0	4m18s
app-7d79574b7c-87n25	1/1	Running	0	4m18s
app-7d79574b7c-8gf8f	1/1	Running	0	4m18s
redis-748b786468-pdklg	1/1	Running	0	11m

Теперь проверим работу нашего приложения. Для этого пробросим порты и сделаем несколько запросов к API:

```
> kubectl port-forward services/app-service 5556:5555
Forwarding from 127.0.0.1:5556 -> 5555
Forwarding from [::1]:5556 -> 5555
Handling connection for 5556
Handling connection for 5556
Handling connection for 5556
Handling connection for 5556
Handling connection for 5556
```

```
lab2 git:master
> curl localhost:5556
{"count":1}

lab2 git:master
> curl localhost:5556
{"count":2}

lab2 git:master
> curl localhost:5556
{"count":3}

lab2 git:master
> curl localhost:5556
{"count":4}

lab2 git:master
> curl localhost:5556
{"count":5}
```

Rolling Update

Для демонстрации обновления сервиса без простоя внесем изменение в исходный код приложения:

```
16
17 @app.route("/")
18 def index():
19     count = redis_client.incr("counter")
20     return jsonify({"count": count, "meta": "version 2"}), HTTPStatus.OK
21
22
```

Теперь наше API возвращает также поле meta

Соберем новую версию приложения, загрузим его в кластер и применим манифесты

```
> docker build -t lab2:v2 .
[+] Building 0.9s (10/10) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 408B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> [internal] load build context
=> => transferring context: 137.42kB
=> CACHED [2/5] WORKDIR /api
=> CACHED [3/5] COPY api/requirements.txt ./
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY api/ ./
=> exporting to image
=> => exporting layers
=> => writing image sha256:53fe0dfd90c2de07dbe0803f622a3415bef2d22e7f0e0fa9320806845104a53b
=> => naming to docker.io/library/lab2:v2
```

```
> kind load docker-image lab2:v2 --name lab2
Image: "lab2:v2" with ID "sha256:53fe0dfd90c2de07dbe0803f622a3415bef2d22e7f0e0fa9320806845104a53b" not yet present on node "lab2-control-plane", loading...

lab2 git:master
> 
```

```
> make env
. .env && for f in k8s/*.yaml; do \
    envsubst < "$f" | kubectl apply -f -; \
done
deployment.apps/app configured
service/app-service unchanged
secret/app-secrets configured
configmap/app-config unchanged
deployment.apps/redis unchanged
service/redis unchanged
```

И выкатим новую версию:

```
> kubectl rollout status deployment/app
deployment "app" successfully rolled out
```

Проверим работу нашего API:

```
lab2 git:master
> curl localhost:5556
{"count":6,"meta":"version 2"}

lab2 git:master
> curl localhost:5556
{"count":7,"meta":"version 2"}

lab2 git:master
> curl localhost:5556
{"count":8,"meta":"version 2"}
```

Заключение

В рамках лабораторной работы выполнена миграция веб-приложения из локальной среды Docker Compose в управляемый кластер Kubernetes. Были изучены и применены основные Kubernetes-объекты: Pods, Deployments и Services. Особое внимание уделялось стратегии Rolling Update, которая позволила обновлять версию приложения без простоев. Работа с декларативными YAML-манифестами наглядно продемонстрировала преимущества такого подхода в управлении целевым состоянием инфраструктуры по сравнению с императивным методом запуска контейнеров.